Re-audit: injective dimension in `ModuleCat`

Lean/API design, categorical pattern, and proof-shape review

Verdict

The dimension-shifting argument is coherent: embed into an injective, identify the two cokernels, and induct. The universe bridge is also a sensible implementation of a genuinely cross-universe statement. I would request two cleanups (dead code and misleading section placement), then approve. The strongest follow-up is not another local simplification: it is reusable transport API for exact equivalences and semilinear quotients.

Findings, with quoted code

1. Remove the dead elaboration line — High

```
have := exactS.hasInjectiveDimensionLT_X₃_iff n inferInstance
apply (exactS.hasInjectiveDimensionLT_X₃_iff n inferInstance).symm.trans
  ((ih e).trans (exactS'.hasInjectiveDimensionLT_X₃_iff n inferInstance))
```

The anonymous `have` is unused, and the identical expression is built on the next line. Delete it. Prefer `exact` rather than `apply` here because the entire proposition is supplied:

```
exact (exactS.hasInjectiveDimensionLT_X₃_iff n inferInstance).symm.trans
  ((ih eCoker).trans
    (exactS'.hasInjectiveDimensionLT_X₃_iff n inferInstance))
```

2. Move the linear auxiliary lemma out of the ring-equivalence section — High

```
variable [Small.{v} R] {R' : Type u'} [CommRing R'] (e : R ≃+* R')

lemma hasInjectiveDimensionLE_iff_of_linearEquiv_aux
    {M : ModuleCat.{v} R} {N : ModuleCat.{max v v'} R}
    (e : M ≃ₗ[R] N) (n : ℕ) : ...
```

This lemma uses neither `R'` nor the ring equivalence from the section. Lean omits unused section parameters from the generated declaration, so this is not a theorem-signature bug; it is a source-structure and readability problem. It also makes the inner `e` shadow the outer `e`. Give this lemma its own section and use names such as `eMN` and `eCoker`.

3. Fix and strengthen the module documentation — High

The original diff's heading says:

```
# Projective Dimension in ModuleCat
```

It must say `Injective Dimension`. Add one terse scope sentence: the file proves invariance of `HasInjectiveDimensionLE` and `injectiveDimension` under linear and ring-semilinear equivalences, including differing universes. This makes the unusual universe machinery discoverable.

4. Factor semilinear equivalences on quotients — Medium

```
let eCoker : ModuleCat.of R (I / f.range) ≃ₛₗ[RingHomClass.toRingHom e]
    ModuleCat.of R' (I' / f'.hom.range) := {
  __ := Submodule.mapQ _ _ eI.toLinearMap
    (by simp [← eqmap, Submodule.map_equiv_eq_comap_symm])
  invFun := Submodule.mapQ _ _ eI.symm.toLinearMap
    (by simp [← eqmap, ← Submodule.map_le_iff_le_comap])
  left_inv x := Submodule.Quotient.induction_on _ x (by simp)
  right_inv x := Submodule.Quotient.induction_on _ x (by simp) }
```

This is correct-looking but low-level and broadly reusable. The existing same-ring `Submodule.Quotient.equiv` pattern suggests adding its semilinear analogue: a semilinear equivalence plus equality of mapped submodules induces an equivalence of quotients. Then this block becomes one constructor call, and inverse proofs stop obscuring dimension shifting.

5. Package the repeated "mono to cokernel short exact sequence" — Medium

```
let S : ShortComplex (ModuleCat.{v} R) :=
  ModuleCat.shortComplexOfCompEqZero _ _
    f.exact_map_mkQ_range.linearMap_comp_eq_zero
have exactS := ModuleCat.shortComplex_shortExact S
  f.exact_map_mkQ_range injf (Submodule.mkQ_surjective _)
```

The proof repeats this shape for `f` and `f'`. A helper returning the canonical short exact sequence $0 \to M \xrightarrow{f} I \to I/\operatorname{range}(f) \to 0$ from `Function.Injective f` would remove bookkeeping and expose the mathematical step. If such a helper belongs in the short-complex API, put it there rather than locally in this file.

6. Name local instances and evidence — Medium

```
have : Module.Injective R' I' :=
  Module.injective_module_of_injective_object R' I'
...
have : Injective (ModuleCat.of R I) := by
  rw [← Module.injective_iff_injective_object]
  exact Module.Injective.of_ringEquiv e.symm eI.symm
```

Anonymous typeclass evidence works, but names such as `injI'` and `injI` make later `inferInstance` calls auditable and avoid fragility if another proposition of the same shape is introduced. Also consider writing `Injective I` if it elaborates definitionally; the extra `ModuleCat.of R I` visually suggests a second object even though `I` is already a `ModuleCat` object.

7. Improve names at the equality boundary — Low

```
refine eq_of_forall_ge_iff (fun N => ?_)
induction N with
```

Here `N` already denotes a module in the theorem statement. Rename the extended-natural cutoff to `d` or `k`. Likewise avoid three meanings of `e`: ring equivalence, module equivalence, and quotient equivalence. Suggested vocabulary: `eR`, `eMN`, `eI`, `eCoker`.

8. Keep the universe bridge; make its purpose explicit — Low

```
let eM : M \simeq_l[R] ModuleCat.of R (ULift.{v'} M) :=
  ULift.moduleEquiv.symm
let eN : N \simeq_l[R'] ModuleCat.of R' (ULift.{v} N) :=
  ULift.moduleEquiv.symm
rw [hasInjectiveDimensionLE_iff_of_linearEquiv_aux eM,
  hasInjectiveDimensionLE_iff_of_linearEquiv_aux eN]
```

This is a good pattern: normalize both objects into $\max(v, v')$, apply the same-universe theorem, and transport back. Do not collapse it into opaque coercion tricks. A short comment saying "move both modules to a common universe" is justified here.

9. Local options and instances are correctly scoped — positive

```
set_option backward.isDefEq.respectTransparency.types false in
attribute [local instance] RingHomInvPair.of_ringEquiv in
lemma hasInjectiveDimensionLE_iff_of_semiLinearEquiv_aux ...
```

The narrow `in` scope is preferable to changing global elaboration behavior. Keep this shape. If later API removes the elaboration workaround, delete the option rather than broadening its scope.

10. Preserve the thin public specialization — positive

```
lemma injectiveDimension_eq_of_linearEquiv (e : M ≃ₗ[R] N) :
    injectiveDimension M = injectiveDimension N :=
  injectiveDimension_eq_of_semiLinearEquiv.{v, v'}
    (M := M) (N := N) (RingEquiv.refl R) e
```

This is good API layering: the linear theorem is a one-line specialization of the stronger semilinear theorem, not a duplicated proof. Keep it. The explicit universe arguments and named module arguments are justified here because they document which two universe levels are being related.

11. Check imports with the project linter — Low

```
public import Mathlib.Algebra.Category.ModuleCat.Ext.DimensionShifting
public import Mathlib.Algebra.Category.ModuleCat.EnoughInjectives
public import Mathlib.Algebra.Category.ModuleCat.Injective
public import Mathlib.Algebra.Homology.ShortComplex.ModuleCat
public import Mathlib.CategoryTheory.Abelian.Injective.Dimension
```

Several may be transitively available, but do not guess from this review: run Mathlib's minimum-import/lint workflow on the target branch and retain only direct conceptual dependencies. The explicit imports are otherwise clear.

Big idea: categorical transport, not module-level reconstruction

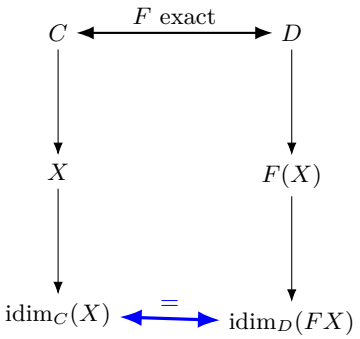
The proof manually realizes an equivalence of module categories induced by $e : R \simeq R'$. Conceptually, an exact equivalence of abelian categories preserves injectives, short exact sequences, and therefore injective dimension. A reusable theorem should have the shape

$$F : C \simeq D, \quad F \text{ exact} \quad \Longrightarrow \quad \operatorname{idim}(X) = \operatorname{idim}(FX).$$

The current theorem then becomes an application to the module-category equivalence associated to a ring equivalence. This would eliminate most of the transported injective and quotient construction:

```
have : Injective (ModuleCat.of R I) := by
  rw [← Module.injective_iff_injective_object]
  exact Module.Injective.of_ringEquiv e.symm eI.symm
...
let eCoker : ModuleCat.of R (I / f.range) \simeq_sl[...] ... := { ... }
```

This is a follow-up design direction, not a blocker: developing the exact-equivalence theorem may be larger than this PR.



Current induction, made visible

$$\begin{array}{ccccccccc} 0 & \longrightarrow & M & \xrightarrow{f} & I & \longrightarrow & C & \longrightarrow & 0 \\ & & \downarrow{\scriptstyle e'}\sim & & \downarrow{\scriptstyle e_I}\sim & & \downarrow{\scriptstyle e_C}\sim & & \\ 0 & \longrightarrow & N & \xrightarrow{f'} & I' & \longrightarrow & C' & \longrightarrow & 0 \end{array}$$

$$\operatorname{idim}(M) \le n+1 \iff \operatorname{idim}(C) \le n \iff \operatorname{idim}(C') \le n \iff \operatorname{idim}(N) \le n+1.$$

This is the right proof invariant. Refactoring should preserve this visible four-step chain.

Proposed order of action

1. Delete the unused `have`; correct the title.
2. Isolate the linear auxiliary section and remove shadowing names.
3. Add a semilinear quotient-equivalence constructor, if acceptable in scope.
4. Optionally package the canonical short exact sequence for an injective linear map.
5. Treat exact-equivalence invariance as a separate categorical follow-up.

Verification note

This audit project is pinned to an older Mathlib snapshot than the supplied diff: several quoted APIs are unavailable locally. Therefore this document reviews the submitted source and proof architecture; it does not claim a local compilation of that newer patch. The review artifact itself was compiled to PDF.